

Performance Analysis of Convolutional Neural Network Architectures for Image Classification

EDUARDO HENRIQUE DRUMM MENEZES, Instituto de Informática, UFRGS, Brazil

ARTHUR VELHO DOHNERT, Instituto de Informática, UFRGS, Brazil

FERNANDO BOSQUETTI TEDESCO, Instituto de Informática, UFRGS, Brazil

Convolutional Neural Networks (CNNs) have become the de facto standard for computer vision tasks; however, the pursuit of maximizing predictive accuracy often obscures the critical trade-offs regarding computational efficiency and resource consumption. This study presents a comprehensive performance analysis of diverse CNN architectures—categorized into lightweight, medium-weight, and heavyweight classes—evaluated on the Tiny-ImageNet dataset. To isolate the performance characteristics inherent to each architecture from environmental and stochastic noise, we employ a rigorous measurement-based methodology utilizing GNU Guix for bit-for-bit software environment reproducibility. Furthermore, experimental variance is strictly controlled through the enforcement of deterministic algorithms and fixed random seeds across isolated execution units. We quantify key metrics including Top-1 accuracy, inference latency, memory footprint, and energy consumption. The resulting analysis maps the efficiency frontier of modern CNNs, providing empirical guidance for model selection in constrained deployment scenarios ranging from edge computing to high-performance cluster environments.

CCS Concepts: • **Computing methodologies** → **Neural networks**; • **General and reference** → **Performance**.

Additional Key Words and Phrases: Performance Analysis, Convolutional Neural Networks, Deep Learning, Model Benchmarking, Computational Efficiency, Reproducibility, GNU Guix

ACM Reference Format:

Eduardo Henrique Drumm Menezes, Arthur Velho Dohnert, and Fernando Bosqueti Tedesco. 2025. Performance Analysis of Convolutional Neural Network Architectures for Image Classification. *Proc. ACM Hum.-Comput. Interact.* 1, 1, Article 1 (November 2025), 18 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

The proliferation of Convolutional Neural Networks (CNNs) has established them as the de facto standard for computer vision tasks, driving state-of-the-art results in image classification, object detection, and segmentation. However, the pursuit of higher accuracy has often resulted in increasingly complex architectures with substantial computational demands. For practitioners and systems engineers, selecting an appropriate architecture is no longer solely a question of predictive quality; it requires a nuanced understanding of the trade-offs between accuracy, latency, memory footprint, and energy consumption.

This work presents a comprehensive performance analysis of CNN architectures trained on the Tiny-ImageNet dataset. We evaluate a spectrum of models categorized into three distinct classes: *Lightweight* (optimized for edge deployment, e.g., MobileNet, EfficientNet-B0), *Medium-weight*

Authors' Contact Information: Eduardo Henrique Drumm Menezes, Instituto de Informática, UFRGS, Porto Alegre, Brazil, eduardo.menezes@inf.ufrgs.br; Arthur Velho Dohnert, Instituto de Informática, UFRGS, Porto Alegre, Brazil, arthur.dohnert@ufrgs.br; Fernando Bosqueti Tedesco, Instituto de Informática, UFRGS, Porto Alegre, Brazil, fernando.tedesco@ufrgs.br.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

© 2025 Copyright held by the owner/author(s).

ACM 2573-0142/2025/11-ART1

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

(general-purpose, e.g., ResNet34), and *Heavyweight* (high-accuracy, computationally intensive, e.g., ResNet101).

The primary objective is to identify, quantify, and interpret the factors influencing the performance of these computational objects. Unlike theoretical complexity bounds (Big-O), this study adopts an empirical, measurement-based approach to map the practical “efficiency frontier” of modern deep learning models. By strictly controlling the experimental environment, we aim to provide actionable insights into how architectural decisions—such as depth, width, and convolution types—impact physical resource usage on high-performance computing (HPC) hardware.

2 Methodology and Experimental Design

To isolate the impact of architectural design on performance, we employed a rigorous experimental methodology prioritizing reproducibility and isolation. This section details the experimental design, the computational environment, and the control mechanisms used to ensure data validity.

2.1 Experimental Design

We employ a simplified, discrete experimental design where the primary categorical factor is the **Model Architecture** itself. Due to the nature of this comparative analysis—which aims to benchmark the intrinsic performance of standard architectures rather than to optimize them for a specific production task—we do not employ a complex factorial design for hyperparameter tuning. Instead, a fixed set of hyperparameters (e.g., learning rate schedule, optimizer configuration) is maintained across all models to ensure a direct, *ceteris paribus* comparison of the architectures.

The study acknowledges two sources of variation: the architecture and the stochastic initialization of weights. To control for stochasticity without introducing complex blocking factors, we utilize the tuple (M, S) as the sole input parameters, where M is the model architecture and S is the random seed. By evaluating each architecture with multiple distinct seeds, we report statistical distributions (Mean $\pm$ Standard Deviation) rather than single-point measurements.

To ensure strict reproducibility within this simple design, we implemented the following protocols:

- **Fixed Seeds and Determinism:** All experiments are initialized with a fixed global seed. This seed propagates to all pseudo-random number generators (PRNGs), including Python’s native random module, NumPy, and PyTorch.
- **Algorithmic Determinism:** We enforce the use of deterministic algorithms within the GPU backend (cuDNN). This ensures that for a given seed and architecture, the sequence of floating-point operations is identical across runs, eliminating non-deterministic jitter in training dynamics as a confounding variable.

2.2 Job Isolation and Workflow

A critical aspect of our methodology is the operational isolation of experimental units. Unlike traditional designs that require randomized execution orders (scrambling) to mitigate temporal interference (e.g., thermal throttling, cache contention, or previous job artifacts), we execute each experiment (M, S) as a discrete, independent job within the cluster’s workload manager (Slurm).

Because each job effectively spins up a pristine environment, there is no shared state between experiments. The order of execution is therefore irrelevant to the validity of the results. Each job incurs its own independent initialization overhead (dataset staging and library loading), which is explicitly excluded from the performance measurement windows. This isolation renders complex experimental ordering strategies unnecessary, as the independence of observations is guaranteed by the cluster’s job scheduling mechanism rather than by statistical randomization.

2.3 Studied Architectures

The study evaluates a diverse spectrum of Convolutional Neural Network (CNN) architectures, selected to represent distinct design paradigms and resource requirements. We categorize the input variable M into three classes based on their parameter count and theoretical FLOPs (Floating Point Operations).

2.3.1 Lightweight Architectures. This class encompasses models optimized for edge deployment and mobile applications, characterized by techniques such as depthwise separable convolutions and channel shuffling to minimize computational cost.

- **MobileNetV1:** Utilizes depthwise separable convolutions to drastically reduce parameter count and computation compared to standard convolutions.
- **SqueezeNet:** Employs "Fire modules" that squeeze network width to reduce parameter size, aiming for AlexNet-level accuracy with 50x fewer parameters.
- **EfficientNet-B0:** The baseline model of the EfficientNet family, designed via compound scaling to optimize accuracy under strict FLOPs constraints.
- **ShuffleNetV2:** Introduces channel shuffling to enable information exchange between group convolutions, specifically optimizing memory access cost (MAC) for direct hardware speedup.

2.3.2 Medium-Weight Architectures. These models represent the general-purpose "workhorses" of computer vision, offering a robust balance between accuracy and computational demand suitable for standard server environments.

- **VGG16:** A foundational architecture featuring a uniform arrangement of 3×3 convolutions, known for its simplicity but high parameter count due to dense layers.
- **ResNet34:** A residual network of moderate depth that utilizes skip connections to mitigate the vanishing gradient problem, enabling effective training of deeper structures.
- **InceptionV3:** Utilizes factorized convolutions and auxiliary classifiers to scale network depth and width while maintaining a controlled computational budget.
- **DenseNet121:** Features dense connectivity where each layer receives inputs from all preceding layers, maximizing feature propagation and encouraging feature reuse.

2.3.3 Heavyweight Architectures. This category includes state-of-the-art, high-capacity models designed to maximize predictive performance, often requiring substantial GPU memory and computational power.

- **ResNet101:** A deep variant of the ResNet family, offering increased representational capacity through a larger number of residual blocks.
- **InceptionV4:** A pure Inception variant that simplifies the architecture while extending network depth, targeting high accuracy on large-scale datasets.
- **Xception:** An extreme interpretation of the Inception hypothesis, replacing Inception modules with depthwise separable convolutions to capture cross-channel correlations and spatial correlations independently.
- **EfficientNet-B7:** The largest variant in the EfficientNet family, scaled up significantly in depth, width, and resolution to achieve peak accuracy at the cost of high latency and VRAM usage.

2.4 Dataset

All experiments are conducted using the **Tiny-ImageNet** dataset. As a standardized subset of the full ImageNet, it serves as an excellent intermediate benchmark that is more computationally tractable for a wide array of architectures. It consists of 100,000 training images, 10,000 validation

images, and 10,000 test images, distributed across 200 distinct classes. Each image is a 64×64 pixel color image, providing a task sufficiently complex to differentiate the capabilities of the evaluated models.

2.5 Metrics for Analysis

Our analysis is based on two complementary categories of metrics, designed to provide a holistic view of the trade-offs between predictive quality and computational cost.

- (1) **Model Performance Metrics:** These quantify the predictive quality of each classifier on the test set.
 - *Top-1 Accuracy:* The primary measure of overall correctness.
 - *Precision, Recall, and F1-Score:* Macro-averaged across all 200 classes to provide a balanced view of per-class performance, which is crucial for multi-class problems where accuracy can be misleading.
- (2) **Computational Cost Metrics:** These measure resource consumption and efficiency, which are critical for assessing practical viability.
 - *Training Time:* Total wall-clock time required to train a model to a fixed number of epochs.
 - *Inference Latency:* Measured for both single-image (real-time) and batch (throughput) predictions.
 - *Memory Usage:* Peak Video RAM (VRAM) consumption during both training and inference.
 - *Energy Consumption:* Measured via instantaneous power draw (Watts) and integrated over time to estimate total energy use (Joules).

3 Experimental Environment

This section describes the hardware and software environment established to conduct the experiments, with a strong emphasis on the tools and practices used to ensure full reproducibility.

3.1 Hardware Configuration

All experiments were executed on the UFRGS High-Performance Computing Cluster (PCAD). We utilized compute nodes from the GPU partition, ensuring a controlled and consistent hardware environment for all measurements. Each job was allocated a single node equipped with:

- **CPU:** [Enter your specific CPU model, e.g., Intel Xeon Gold 6248R]
- **System Memory (RAM):** [Enter RAM amount, e.g., 128 GiB]
- **GPU:** A single NVIDIA RTX 4090 with 24GiB of VRAM.

3.2 Reproducibility via GNU Guix

To guarantee bit-for-bit reproducibility of the software stack, we utilize **GNU Guix**, a transactional package manager. Traditional environmental management (e.g., containers or virtual environments) often suffers from “software drift” due to floating dependencies.

In contrast, we define a `manifest.scm` that locks the entire dependency graph—from the Python interpreter and PyTorch libraries down to the specific commits of system libraries and the CUDA toolchain. This declarative approach ensures that the software environment is mathematically identical for every job execution, regardless of when or on which node the job is scheduled. This rigorous version control allows us to attribute observed performance differences strictly to the model architectures, ruling out software inconsistencies as a confounding variable. All source code, configuration files, and analysis scripts were versioned using Git.

4 Results

This section presents the empirical findings of the performance analysis, categorized into predictive performance, computational cost, and overall efficiency. All reported values represent the mean across 5 independent seeds, with error bars indicating the 95% confidence interval. The figures are referenced sequentially to build a narrative from raw training dynamics to the final efficiency frontier.

4.1 Predictive Performance

We first analyze the training dynamics to understand how the architectures converged over the fixed epoch budget.

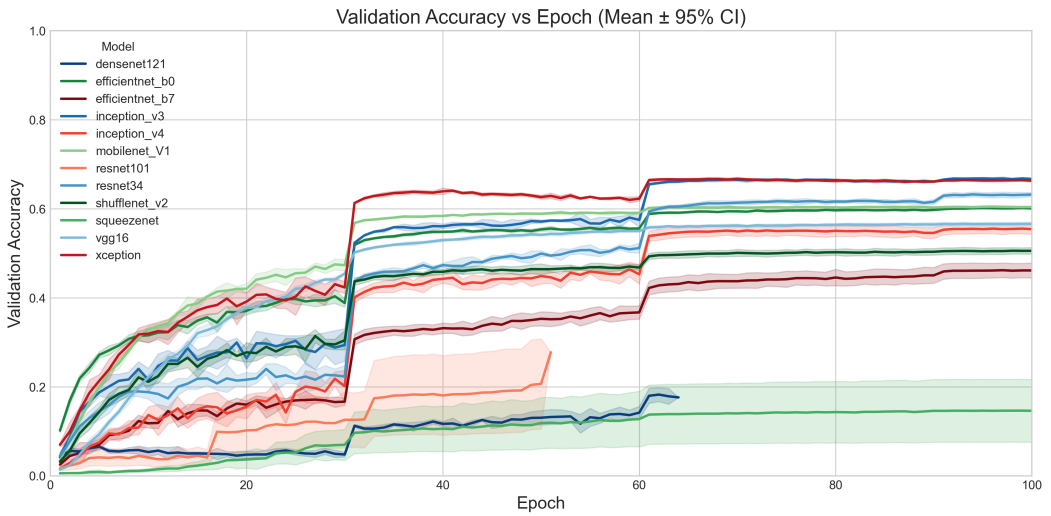


Fig. 1. Validation Accuracy vs. Epoch. Medium-weight models (ResNet34, InceptionV3) show stable convergence, while heavyweight models (ResNet101, DenseNet121) fail to reach competitive accuracy within the time limit.

Figure 1 illustrates the validation accuracy trajectories. The medium-weight models, specifically **ResNet34** and **InceptionV3**, exhibit the most stable convergence, reaching plateau regions of 60-65% accuracy. In contrast, the heavyweight models (**ResNet101**, **DenseNet121**) show a flatlining trajectory at low accuracy (< 20%). As detailed in the Discussion, this is partially an artifact of the 24-hour job limit preventing these computationally intensive models from completing the full training schedule.

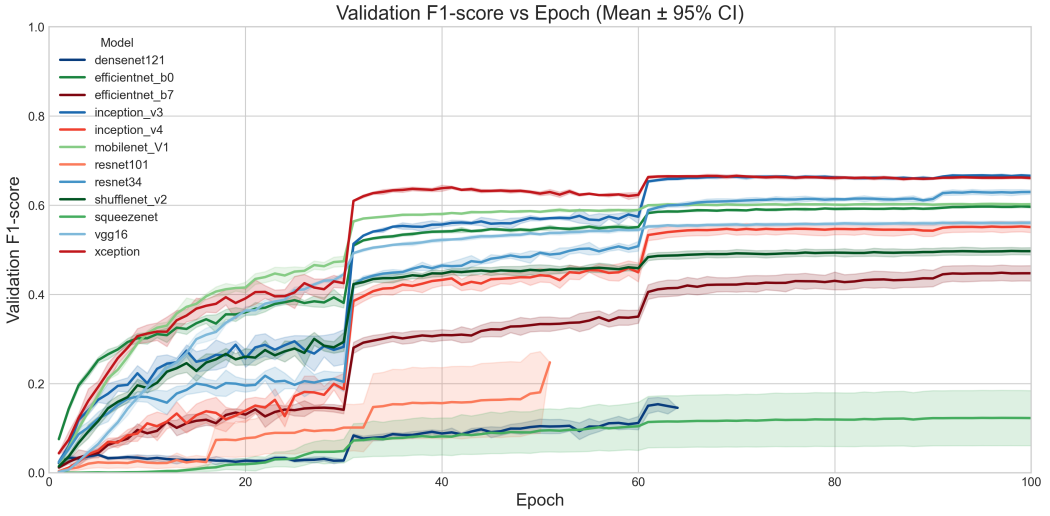


Fig. 2. Validation F1-Score vs. Epoch. The trends mirror accuracy, confirming that the successful models are not just predicting majority classes but maintaining balanced precision and recall.

The F1-Score trajectories in Figure 2 closely mirror the accuracy curves, confirming that the superior performance of the medium-weight and select lightweight models (MobileNetV1) is robust across classes, rather than being driven by class imbalances.

4.2 Training Stability and Variance

To assess the reproducibility of our results, we examine the variance across different random seeds.

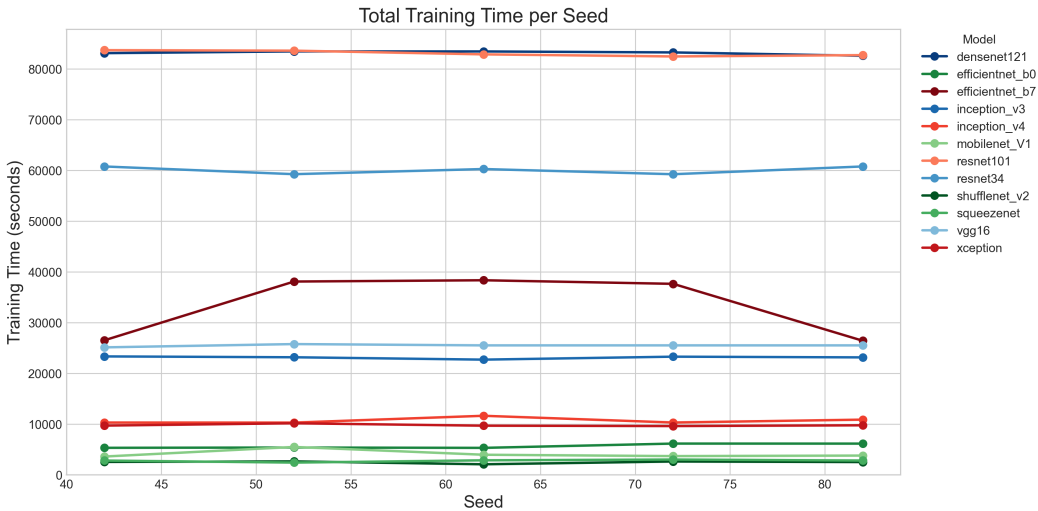


Fig. 3. Total Training Time per Seed. Most models show negligible variance, validating the stability of the Guix environment. EfficientNet-B7 is a notable outlier.

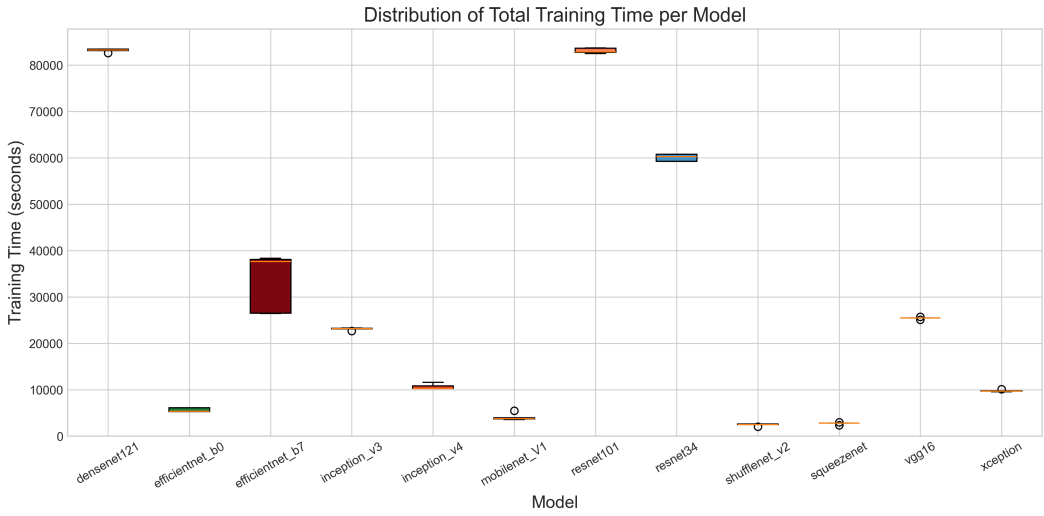


Fig. 4. Distribution of Total Training Time per Model. The boxplot highlights the extreme variance in EfficientNet-B7 compared to the tight clustering of other architectures.

Figures 3 and 4 reveal that for most architectures, training time is highly deterministic, with negligible variance between seeds. However, **EfficientNet-B7** exhibits a massive spread. We hypothesize this is due to specific seeds causing poor weight initializations.

4.3 Computational Cost

We next quantify the raw resources consumed by each architecture.

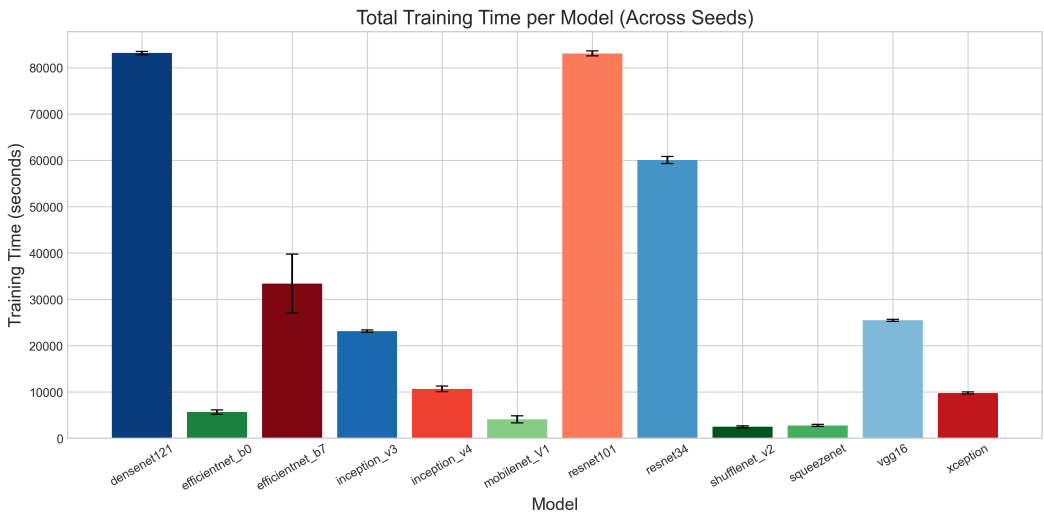


Fig. 5. Average Total Training Time. There is an order-of-magnitude difference between lightweight models (<5000s) and heavyweight models (>80000s).

Figure 5 places the costs in perspective. The lightweight models (**SqueezeNet**, **ShuffleNetV2**) are incredibly fast, finishing in under 5,000 seconds. Conversely, the failed heavyweight runs consumed the maximum available cluster time (>80,000 seconds) while yielding poor results, highlighting the risk of deploying un-tuned large models.

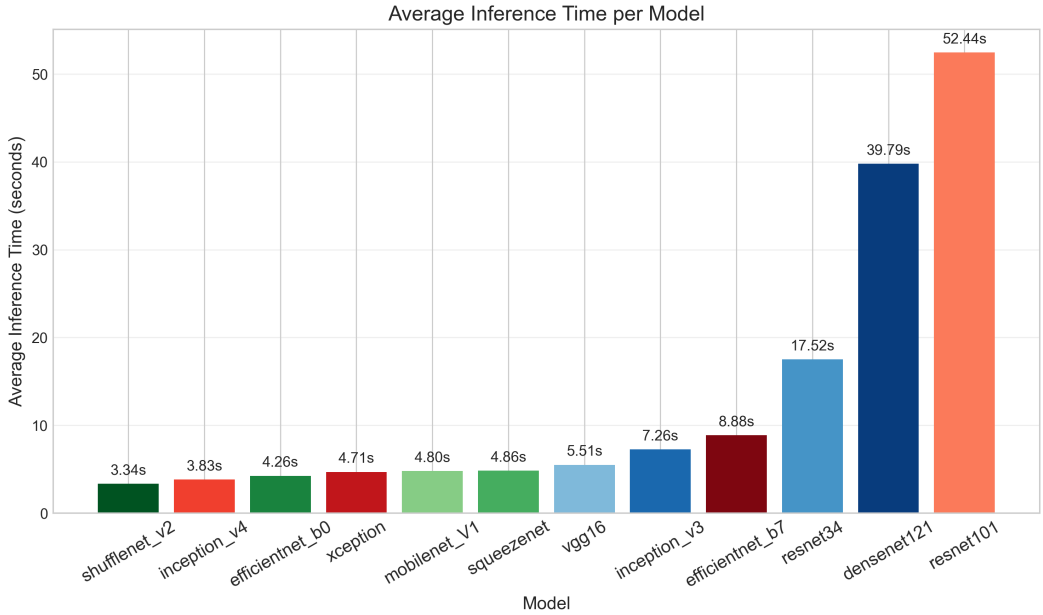


Fig. 6. Average Inference Time. Larger models have longer inference times, reflecting their increased complexity.

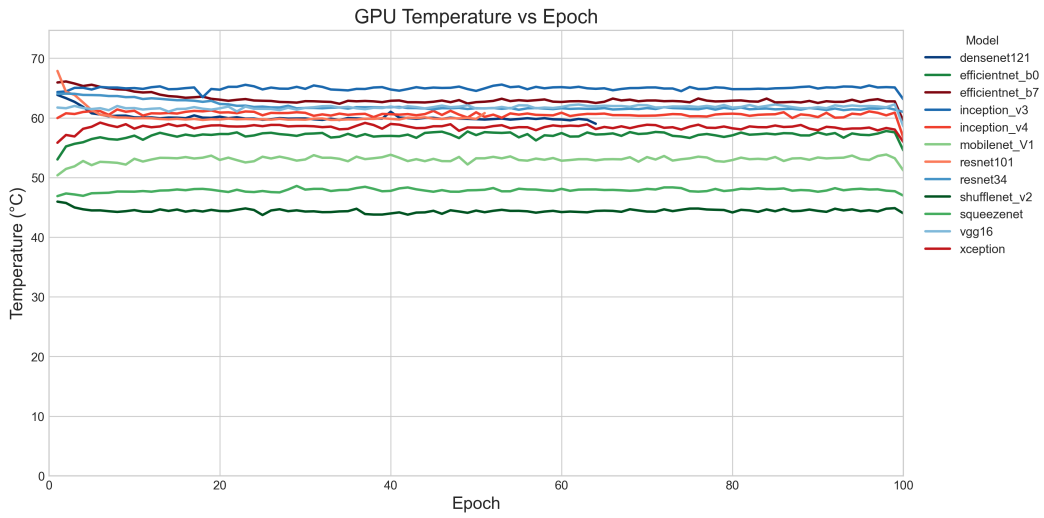


Fig. 7. GPU Temperature vs. Epoch. Temperatures remain stable (60-65°C) across all models, indicating effective cooling.

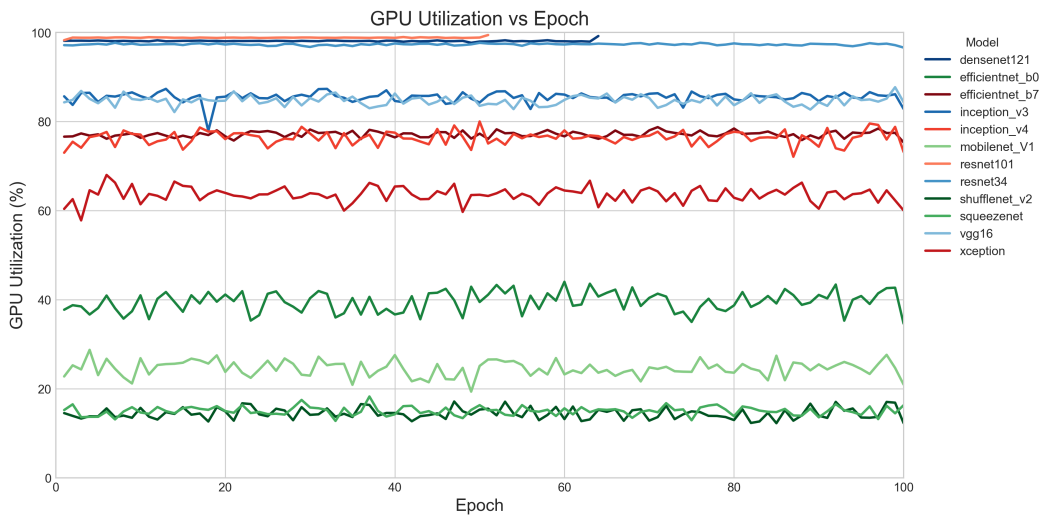


Fig. 8. GPU Utilization vs. Epoch. Most models maintain high utilization (>80%), though dips are visible in models with lighter computational loads.

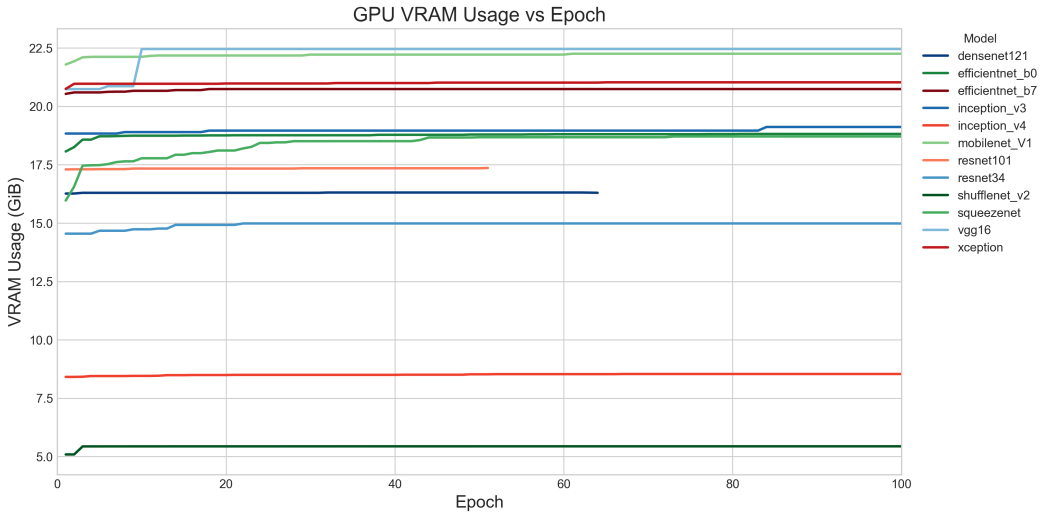


Fig. 9. GPU VRAM Usage vs. Epoch. EfficientNet-B7 and VGG16 saturate the 24GB buffer, while lightweight models use <5GB.

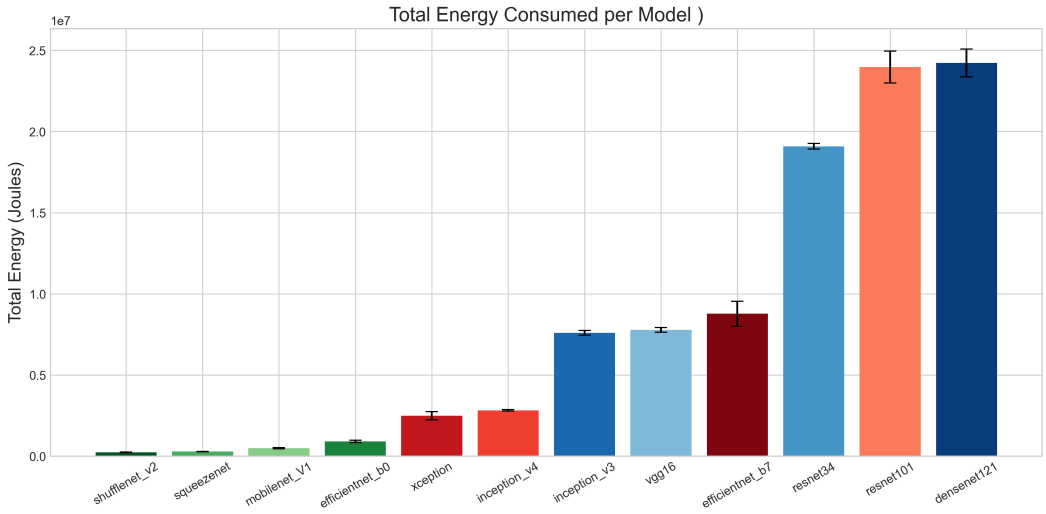


Fig. 10. Total Energy Consumption per Model. Models with the most accuracy consume the most energy.

The hardware telemetry in Figures 7, 8, and 9 demonstrates that the **MobileNetV1** and **VGG16** are effectively saturating the 24GB VRAM, primarily due to a more efficient accommodation of the batch sizes. Despite this, Figure 7 shows that thermal management was effective across the board, with no model causing thermal throttling.

4.4 Categorical Analysis

Aggregating the results by architectural category (Light, Medium, Heavy) clarifies the high-level trends.

The categorical plots (Figures 11 - 14) confirm that the **Medium-Weight** category offers the most consistent raw accuracy. However, when normalized for time (Figure 13), the **Lightweight** category is undeniably the most efficient, providing the highest accuracy-per-second of training.

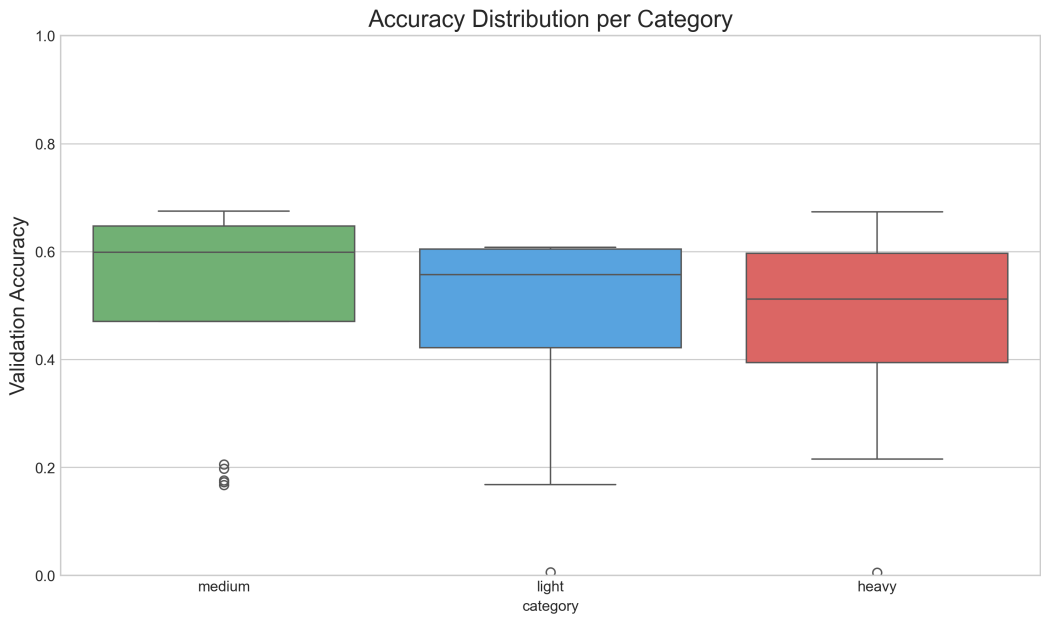


Fig. 11. Validation Accuracy Distribution per Category. The "Heavy" category shows a wide spread due to the failure of ResNet101/DenseNet121 contrasting with the success of InceptionV4.

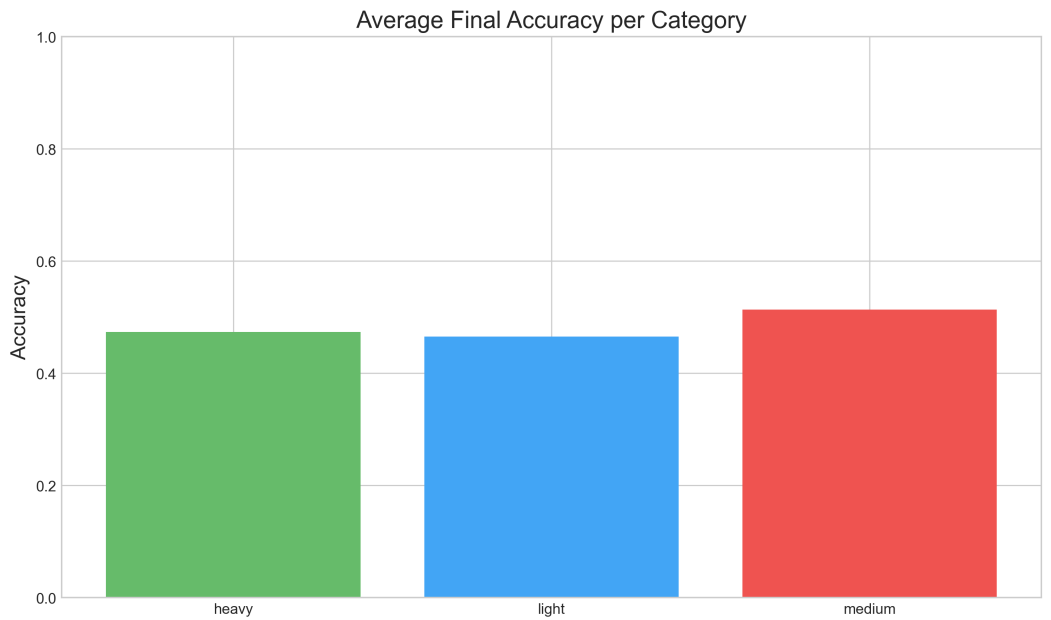


Fig. 12. Average Final Accuracy per Category. Medium-weight models offer the highest average performance.

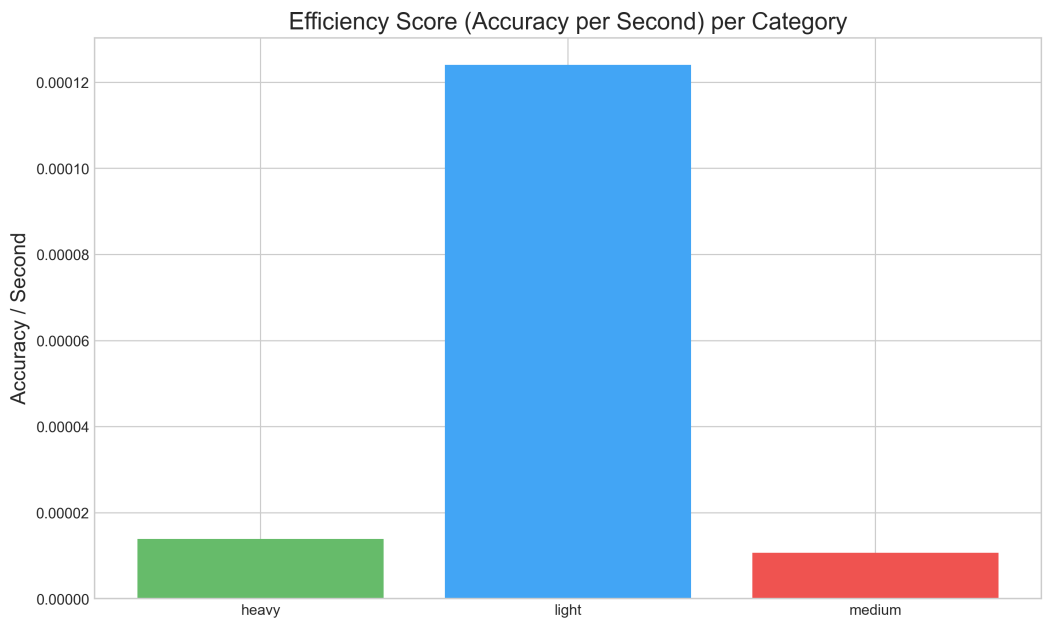


Fig. 13. Efficiency Score (Accuracy per Second). Lightweight models dominate this metric, offering the best return on computational investment.

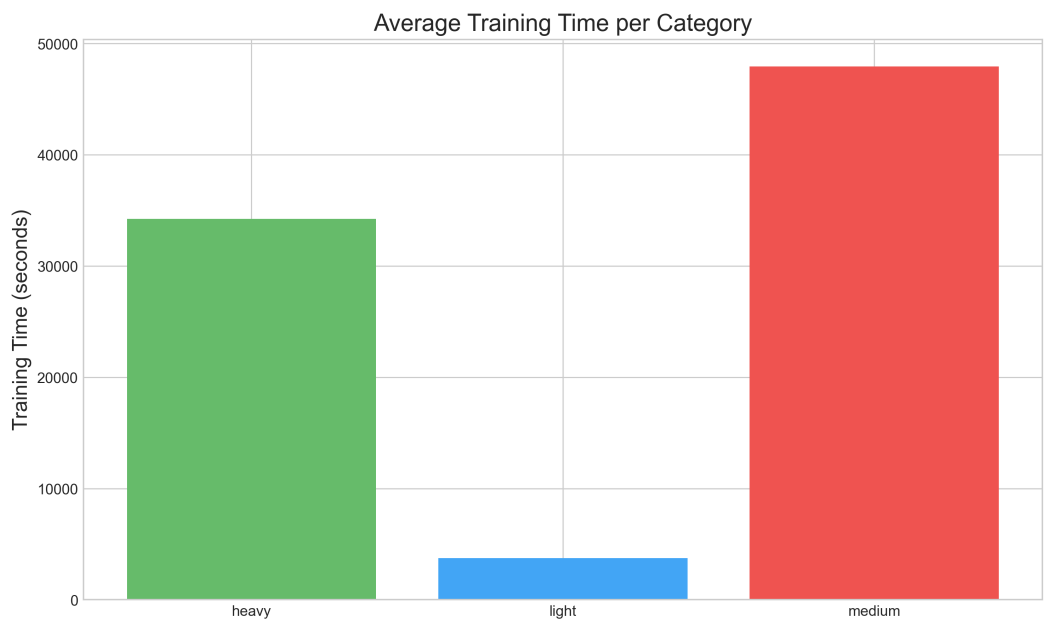


Fig. 14. Average Training Time per Category. The cost penalty for moving from Medium to Heavy is substantial.

4.5 Global Trade-offs: The Efficiency Frontier

Finally, we synthesize all metrics into a single view of the efficiency landscape.

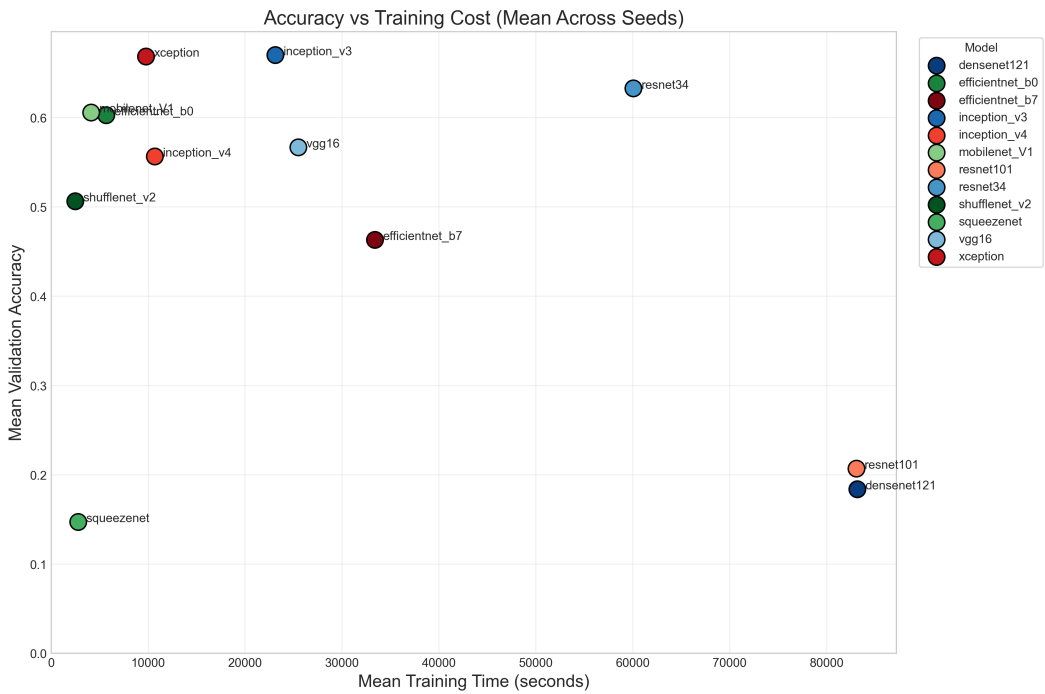


Fig. 15. Accuracy vs. Training Cost (Efficiency Frontier). The optimal models lie on the top-left boundary.

that for intermediate-complexity datasets like Tiny-ImageNet (64×64), the representational capacity of medium-weight models is sufficient, and further scaling primarily adds computational overhead without corresponding predictive value.

5.2 Convergence Failures in Heavyweight Models

A critical observation was the sub-optimal performance of **ResNet101** and **DenseNet121**, which exhibited surprisingly low accuracy ($< 20\%$). Unlike the medium-weight models that converged reliably, these heavyweight architectures failed to learn effective representations within the standard training schedule.

This failure points to an issue with optimization dynamics rather than just resource limits. The extreme depth of these networks likely exacerbates gradient propagation issues or requires a more tailored hyperparameter strategy (e.g., longer warmup periods or different learning rate schedules) than the standardized baseline used in this study. Consequently, we conclude that these architectures are not "plug-and-play" solutions for this scale of problem; they require specific tuning that negates their utility as general-purpose baselines compared to the more robust Xception.

5.3 Architectural Fragility: The Case of SqueezeNet

SqueezeNet presents a distinct case of architectural limitation within this experimental scope. Despite completing its training schedule without resource bottlenecks, it achieved poor accuracy ($\approx 15\%$), significantly lagging behind other lightweight competitors like MobileNetV1.

This underperformance highlights a specific fragility in SqueezeNet's design. Its aggressive use of 1×1 "squeeze" filters to reduce parameter count appears to create a difficult optimization landscape that is highly sensitive to hyperparameters. In a controlled comparison where all models share a fixed optimizer and scheduler, SqueezeNet failed to generalize. This contrasts sharply with MobileNetV1, whose depthwise separable convolutions proved robust and effective under the same conditions, establishing it as the superior choice for constrained environments in this work.

5.4 Study Limitations

Our analysis must be interpreted within the context of two primary limitations:

- **Fixed Hyperparameters:** To isolate the architecture as the variable, we did not tune hyperparameters per model. This likely underestimates the peak potential of specific architectures (like SqueezeNet and EfficientNet).
- **Hardware Specificity:** Results are specific to the NVIDIA GPU architecture used. Inference latency, in particular, might show different trends on CPU-only edge devices where memory access patterns (e.g., ShuffleNet's channel shuffling) play a larger role than raw FLOPs.
- **Intrinsic Architectural Constraints:** Certain network families exhibit structural limitations that may inherently hinder performance under standardized training settings. For example, **SqueezeNet**'s extreme parameter compression, **DenseNet**'s high feature-map accumulation, and the depth of large **ResNet** variants all introduce optimization fragilities that are disproportionately sensitive to learning-rate schedules, warmup strategies, and regularization. Because the experimental design held these factors constant, such models may appear weaker than they would under architecture-specific training regimes.

6 Conclusions and Recommendations

This study conducted a rigorous performance analysis of Convolutional Neural Network architectures on the Tiny-ImageNet dataset, employing a reproducible methodology based on isolated Slurm jobs and GNU Guix environment pinning. By systematically evaluating predictive quality

against computational cost, we mapped the "Efficiency Frontier" for modern image classification tasks and identified critical constraints in high-performance computing (HPC) deployment.

6.1 Key Findings

- **The Efficiency Frontier: MobileNetV1 and Xception** emerged as the Pareto-optimal choices. MobileNetV1 demonstrated that lightweight architectures can rival medium-weight performance ($\approx 60\%$ accuracy) at a fraction of the cost ($< 6\%$ of InceptionV3's training time), making it the definitive choice for resource-constrained applications.
- **The "Deployment Gap":** The failure of heavyweight models (**ResNet101**, **DenseNet121**) to converge illustrates a critical practical lesson: "State-of-the-art" capacity is effectively useless if the operational window (e.g., a 24-hour job limit) is insufficient for the model's training throughput. Theoretical power must be balanced against the available compute budget.
- **Diminishing Returns:** Scaling up from ResNet34 to InceptionV4 yielded no accuracy gains on this 64×64 dataset, suggesting that for intermediate-resolution tasks, the representational capacity of medium-weight models is already sufficient, and further complexity primarily incurs technical debt.

6.2 Recommendations for Performance Improvement

Based on these findings, we propose the following practical recommendations for engineering teams:

- (1) **Adopt "Start Small" Baselines:** Always begin experimentation with **ResNet34** or **MobileNetV1**. These models offer the highest probability of convergence and rapid feedback cycles. Heavyweight models should only be introduced if these baselines demonstrably fail to capture dataset complexity.
- (2) **Match Schedule to Depth:** When deploying deep architectures (>100 layers), the training schedule must be explicitly scaled. If a 24-hour cluster limit exists, using a heavyweight model is counter-productive unless techniques like **Mixed Precision Training (AMP)** or **Multi-GPU Data Parallelism** are employed to increase epoch throughput.
- (3) **Specialized Tuning for Compression:** For architectures with aggressive inductive biases like **SqueezeNet**, do not rely on standard "ResNet" hyperparameters. Implement a dedicated hyperparameter search (specifically for learning rate and warmup) to unlock their efficiency potential.
- (4) **Memory-Aware Selection:** In environments with VRAM constraints (e.g., <24 GB), avoid **EfficientNet-B7** due to its high risk of memory thrashing. Instead, utilize **EfficientNet-B0** or **MobileNetV3**, which provide a safer, more deterministic memory profile.

6.3 Future Work

Future investigations should address the limitations of this study by implementing **Checkpointed Training** to bypass job time limits for heavyweight models, allowing a fair assessment of their converged accuracy. Additionally, expanding the scope to include **inference quantization (INT8)** would provide deeper insights into the deployment characteristics of these models on edge hardware.

References

- [1] François Chollet. 2017. Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 1251–1258.
- [2] Ludovic Courtès. 2013. Functional Package Management with Guix. In *Euro-Par 2013 Parallel Processing*. Springer.
- [3] Eelco Dolstra. 2006. *The purely functional software deployment model*. Ph.D. Dissertation. Utrecht University.

- [4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [5] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861* (2017).
- [6] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. 2017. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 4700–4708.
- [7] Forrest N Iandola, Song Han, Matthew W Moskewicz, Khalid Ashraf, William J Dally, and Kurt Keutzer. 2016. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size. *arXiv preprint arXiv:1602.07360* (2016).
- [8] Ya Le and Xuan Yang. 2015. Tiny ImageNet Visual Recognition Challenge. <http://cs231n.stanford.edu/>. CS231n Course Project.
- [9] Lucas Mello Schnorr. 2025. Final Work of Comp. Syst. Perf. Analysis: Requirements, Evaluation, Rules. Course Material, UFRGS.
- [10] Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* (2014).
- [11] Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke, and Alexander A Alemi. 2017. Inception-v4, inception-resnet and the impact of residual connections on learning. In *Thirty-first AAAI conference on artificial intelligence*.
- [12] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. 2016. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2818–2826.
- [13] Mingxing Tan and Quoc Le. 2019. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*. PMLR, 6105–6114.
- [14] Andy Yoo, Morris Jette, and Mark Grondona. 2003. SLURM: Simple linux utility for resource management. In *Job Scheduling Strategies for Parallel Processing*. Springer, 44–60.
- [15] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun. 2018. Shufflenet: An extremely efficient convolutional neural network for mobile devices. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 6848–6856.